

System Programming - Crystal Campus

W. Busch, J. Perez Centeno, J. Romera

20 de abril de 2026

Índice

1. Ejercicio 1	1
2. Ejercicio 2	3
3. Ejercicio 3	4
4. Ejercicio 4	4
5. Ejercicio 5	5
5.1. mmu_init	6
5.2. mmu_initTaskDir	6
5.3. mmu_mapPage	8
5.4. mmu_unmapPage	9
5.5. Prueba de paginación	10
6. Ejercicio 6	10
6.1. tss_init	10
6.2. fill_tss_idle	11
6.3. fill_tss	12
6.4. Salto de la tarea inicial a la tarea idle	13
7. Ejercicio 7	13
7.1. Inicialización de las estructuras del scheduler	13
7.2. sched_nextTask	14
7.3. Interrupciones de las tareas	15
7.3.1. game_move	16
7.3.2. game_take	18
7.3.3. game_get_id	18
7.4. Intercambio de tareas	18
7.5. Atención de excepciones	19
7.5.1. debugging	19
8. Despacho de tareas	20
8.0.1. Dibujo de pantalla	21

1. Ejercicio 1

En la primera parte del ejercicio 1 completamos la GDT con su configuración inicial. Por instrucción del TP, las primeras 17 posiciones se consideran utilizadas. Por lo tanto, al inicializar la tabla, los índices de los segmentos que vamos a definir van del 18 al 22. Para eso, en el archivo *defines.h* agregamos las constantes:

```
#define GDT_IDX_NULL_DESC 0
#define GDT_DATOS_KERNEL 18
#define GDT_DATOS_USER 19
#define GDT_CODIGO_KERNEL 20
#define GDT_CODIGO_USER 21
#define GDT_SCREEN 22
```

Luego, en el archivo *gdt.c*, agregamos cuatro entradas, correspondientes a sendos segmentos de código y de datos para usuario y para kernel. Completamos los atributos de esas estructuras de acuerdo a sus propiedades.

Como los segmentos de kernel o usuario debían direccionar 66 MB de memoria, los límites de los segmentos son 0x41FF con granularidad 1. La granularidad significa que en realidad la unidad de medida es 4 KB, i.e., 0x1000. Además, los doce dígitos menos significativos no se testean para chequear el offset, y por lo tanto se setean en 1. Eso nos da un límite igual a $0x41FF * 4KB + 0xFFF = 0x41FFFFFF + 0xFFF = 0x41FFFFFF$, es decir, 66MB - 1.

Para los segmentos de kernel pusimos 0x0 en el bit de DPL, y para los segmentos de usuario, 0x3. Además los segmentos de datos tienen el atributo de Type igual a 0010 (es decir, Type = R/W) y los de código igual a 1000 (es decir, Type = Exec only). Por otra parte claramente deben tener el bit de Present seteado.

Para facilitar la lectura de las constantes, usamos las macros:

```
#define LIMIT_00_15(addr) (addr & 0xFFFF)
#define LIMIT_16_19(addr) ((addr >> 16) & 0xF)
#define BASE_00_15(addr) (addr & 0xFFFF)
#define BASE_16_23(addr) ((addr >> 16) & 0xFF)
#define BASE_24_31(addr) ((addr >> 24) & 0xFF)
```

Estas macros aceptan como input una *addr* (address) y nos dan los bits correspondientes. Por ejemplo, *LIMIT_00_15(addr)* retorna los 16 bits menos significativos de *addr*, en el mismo orden.

El quinto segmento que definimos es el de manejo de pantalla. Los atributos de este segmento se completan con los datos provistos por la cátedra: se provee la base y el límite del segmento. Estas constantes las guardamos en el archivo *define*:

```
#define VIDEO          0x000B8000
#define VIDEO_LIMIT    (2 * VIDEO_FILS * VIDEO_COLS - 1)
```

Como queremos escribir y leer los píxeles de la pantalla, el atributo Type lo seteamos como R/W, y el atributo DPL en 0x00 (i.e., nivel de protección correspondiente a kernel), para que sólo este pueda escribir en pantalla.

Para pasar a modo protegido y setear el stack del kernel, seguimos las instrucciones provistas por la cátedra:

- Las instrucciones deshabilitadas y el cambio al modo de video correcto (correspondiente a la resolución de nuestro juego) ya estaban provistos por la cátedra.
- Habilitamos A20.
- Cargamos la GDT con el comando *lgdt* y la dirección de memoria correspondiente al comienzo de la estructura de la GDT.
- Habilitamos el bit PE del registro CR0. Como la única forma de hacerlo es a través del registro EAX, primero movemos CR0 a EAX, luego con un **or** seteamos el bit menos significativo de EAX en 1, y luego movemos EAX a CR0.

- Finalmente, para pasar a modo protegido, la siguiente línea de código que ejecutemos debe estar dentro del segmento correspondiente a código del kernel. El índice en la GDT de este segmento es 20. Además los primeros tres dígitos del selector de segmento son 0, ya que el bit de TI (Table Indicator) es 0, pues es un selector de GDT (sería 1 si fuera de LDT), y el RPL (Requested Privilege Level), también es 0. Entonces queda $10100000 = 1010000 = 0xA0$. Por lo tanto, al hacer

```
jmp 0xA0:modo_protegido
```

ya estamos dentro del código del kernel con segmentación habilitada.

- Luego cargamos los registros de segmentos. Cargamos todos los segmentos de datos con el segmento correspondiente a datos del kernel, excepto el registro fs, que cargamos con el selector correspondiente al segmento de la pantalla en memoria.
- movemos a los registros de stack pointer y base pointer a 0x27000, al ser la dirección requerida por la cátedra.

Finalmente, implementamos una función en C que se encarga de limpiar la pantalla y mostrar un tablero provisorio, que luego desarrollamos en detalle. Para llamar las funciones implementadas en C utilizamos

```
call screen_limpiar  
call screen_draw
```

2. Ejercicio 2

Para este ejercicio utilizamos como punto de partida la macro de ejemplo proporcionada.

Cambiamos los atributos de la estructura IDT_ENTRY para que el selector de segmento sea el de código de kernel, y los atributos los seteamos en 0x8E00: 1000111000000000.

Attribute	Value
Segment	CÓDIGO KERNEL
P	1
DPL	00
S	0
Type	111

Luego agregamos a mano, en la función idt_init(), las IDT_ENTRY correspondientes a las primeras 19 interrupciones. Luego agregamos la entrada por interrupción de reloj, de teclado, y en el lugar 47, interrupción de software.

Además agregamos un array de mensajes para mostrar cuando suceden las interrupciones, para comprobar que efectivamente se están ejecutando.

En el archivo isr.asm inicializamos la rutina de atención de interrupciones para las 255 posibles interrupciones. Lo hicimos modificando la macro `_is %1` para llamar a `idt_print_nro`, que lo que hace simplemente es imprimir el mensaje correspondiente del array de mensajes.

Además declaramos las rutinas de atención de interrupciones en isr.h.

Finalmente, en kernel.asm, inicializamos la IDT y la cargamos en kernel.asm:

```
call idt_init  
  
; Cargar IDT  
lidt [IDT_DESC]
```

Para comprobar el funcionamiento de interrupciones, probamos con una división por cero y efectivamente ejecutó la interrupción.

3. Ejercicio 3

En el ejercicio 3 implementamos las entradas 32, 33 y 47 de la IDT. Para ello definimos rutinas de atención de interrupciones en assembler en el archivo isr.asm.

```
;Rutina atencion interrupcion Clock
global _isr32
_isr32:
call nextClock
call pic_finish1
iret

;Rutina atencion interrupcion teclado
global _isr33
_isr33:
in al, 0x60
push eax
call idt_int_teclado
pop eax
call pic_finish1
iret

global _isr47
_isr47:
mov eax, 0x42
iret
```

La interrupción de reloj, `_isr32`, llama a `nextClock`, `picfinish` y luego hace `iret`. Las función `nextClock` ya venía programada por la cátedra, y lo que hace es hacer avanzar el reloj que se muestra en pantalla. La función `picfinish` notifica al controlador de interrupciones que la interrupción fue atendida. Esto es común en todas las rutinas de atención que implementamos: avisar primero que se atendió a la interrupción.

Para la interrupción de teclado (`_isr33`), creamos una función `idt_int_teclado` que recibe el scan code de la interrupción y lo convierte a un int. Para lograr una rápida comprobación del funcionamiento (o no) del código, hicimos que lo único que hiciera con ese int fuera imprimirlo en pantalla. Posteriormente implementamos una respuesta acorde a si se presionó una tecla direccional, un botón de generar nueva tarea, etc.

En cuanto a la interrupción 47, como dice el ejercicio, por el momento lo único que hace es mover el entero `0x42` al registro `eax`, posponiendo para más adelante su implementación.

4. Ejercicio 4

En el ejercicio 4 debíamos inicializar el directorio y las tablas de páginas para el kernel y activar paginación.

Para eso definimos algunas variables en el archivo `defines.h`:

```
#define PAGE_SIZE      0x00001000
#define FREE_KERNEL_PAGES_START  0x00100000
#define FREE_TASKS_PAGES_START   0x00400000
```

El area libre de páginas del kernel empieza en la dirección `0x00100000`, y el area libre para las tareas, en `0x00400000`. Además definimos como constante el tamaño de cada página (4 kb).

La función `mmu_initKernelDir` mapea los primeros 4 Mb de memoria física (desde `0x000000` a `0x0FFFFFF`) y setea esas páginas como presentes, para indicar que están ocupadas (por el kernel).

Para implementarla, definimos como constante el lugar donde se va a encontrar el directorio de páginas y la page table cero del kernel.

```
#define KERNEL_PAGE_DIR      0x00027000
#define KERNEL_PAGE_TABLE_0 0x00028000
```

Primero definimos en el índice cero del Page Directory del kernel el descriptor de Page Table:

```
uint32_t* kernel_page_dir = (uint32_t*) KERNEL_PAGE_DIR;
kernel_page_dir[0] = KERNEL_PAGE_TABLE_0 | PAG_RW | PAG_P;
```

Luego iteramos por la Page Table inicializando cada entrada con la dirección física igual a la virtual:

```
190  uint32_t* kernel_page_table = (uint32_t*) KERNEL_PAGE_TABLE_0;
192  for (uint16_t i = 0; i < 1024; ++i) {
193      kernel_page_table[i] = (i << 12) | PAG_RW | PAG_P;
194  }
```

En la línea 194 usamos un shift a izquierda del `uint16_t i` ya que en una Page Table Entry, la dirección de la base de una página se encuentra en los 20 dígitos más significativos de la dirección de 32 bits, correspondiendo los primeros 12 dígitos, a los atributos de la entrada.

Finalmente, para activar paginación lo hicimos en tres pasos:

```
; Inicializar el directorio de paginas
call mmu_initKernelDir
```

Llama a la función recién implementada que construye las estructuras de Page Directory y Page Table del kernel.

```
; Cargar directorio de paginas
mov eax, KERNEL_PAGE_DIR
mov cr3, eax
```

Carga en CR3 el directorio de páginas del kernel, ya que empezamos ejecutando desde el código del kernel.

```
; Habilitar paginacion
mov eax, cr0
or eax, CR3_ACTIVATE_PAGING
mov cr0, eax
```

Para activar paginación, seteamos el bit de paginación en el registro CR0.

5. Ejercicio 5

En el ejercicio 5 nos encargamos de implementar las funciones y estructuras restantes encargadas de administrar la memoria.

5.1. mmu_init

La rutina de inicialización es sencilla: simplemente setea las siguientes páginas libres del kernel y de las tasks como las páginas en donde comienzan las páginas libres.

```
void mmu_init() {
    next_free_kernel_page = FREE_KERNEL_PAGES_START;
    next_free_task_page = FREE_TASKS_PAGES_START;
}
```

Además definimos métodos de conveniencia que nos devuelven las próximas páginas libres de tareas y de kernel. Sus implementaciones consisten en simplemente agregar la constante PAGE_SIZE, que representa el tamaño de una página, a la dirección de la página actual.

5.2. mmu_initTaskDir

Para implementar mmu_initTaskDir le agregamos dos variables, dos números de 32 bits sin signo que representan el jugador y la fila actual de la tarea.

En el código de la unidad de administración de páginas definimos las siguientes constantes:

```
#define MMU_BASE_ADDR_TAREAS 0x10000
```

MMU_BASE_ADDR_TAREAS es la posición base de la copia original del código de las tareas. Con un offset podemos obtener el código de la tarea de cada jugador: primero está el código de la tarea del jugador A, luego la del jugador B. Cada tarea ocupa 8KB.

```
#define MMU_BASE_ADDR_AREA_COMPARTIDA 0x15000
```

MMU_BASE_ADDR_AREA_COMPARTIDA es la posición base de las áreas compartidas de las tareas. Como antes, con un offset podemos obtener las áreas libres de las tareas de cada jugador. Cada área ocupa 8Kb.

```
uint32_t addr_codigo_task_original =
    MMU_BASE_ADDR_TAREAS + jugador * 2 * PAGE_SIZE;
```

Buscamos en la memoria física el address base del código de la tarea. Si el jugador es el A (jugador 0), obtenemos la dirección base, si el jugador es el B (jugador 1), le sumamos un offset de 2 páginas.

```
uint32_t addr_area_compartida_jugador =
    MMU_BASE_ADDR_AREA_COMPARTIDA + jugador * 2 * PAGE_SIZE;
```

Hago lo mismo para la dirección del área compartida.

```
uint32_t columna = (jugador == PLAYER_A) ? 0 : 77;
uint32_t indice_tablero = TABLERO_N_COLS * fila + columna;
uint32_t phys_dst_codigo_tarea =
    MAPA_BASE + 2 * PAGE_SIZE * indice_tablero;
```

En esta sección del código calculamos la posición física donde debemos copiar el código de la tarea. El lugar de la tarea se calcula en base al lugar de comienzo de la tarea en el tablero, es decir, de la fila y de la columna de inicio. Primero hallamos la columna. Esta puede ser 0 o 77, dependiendo de si es el

jugador A o B. Para hallar el índice en el tablero usamos la fila pasada como argumento. Como el tablero en memoria es continuo, primero multiplicamos el tamaño de una fila (su cantidad de columnas) por la fila en la que está. Eso me da el índice, módulo la columna. Luego debemos sumarle 0 o 77 dependiendo si es el jugador A o B.

Luego para determinar la dirección física, a la dirección de la base del mapa, le sumamos el índice por el tamaño de cada código (8 Kb).

```
page_dir_entry_t* dir = (page_dir_entry_t*) mmu_nextFreeKernelPage();
for (uint32_t i = 0; i < 1024; i++) {
    dir[i].p = 0; // Ausente
}
```

Una vez determinada la dirección física donde vamos a copiar el código de la tarea, creamos un Page Directory para esta. El primer paso es pedir una página del área libre del kernel donde poner el PD. Luego inicializamos ese PD con todas sus entradas ausentes.

```
238 for (uint32_t i = 0; i < 1024; ++i) {
244     uint32_t attrs = PAG_P | PAG_RW | PAG_S;
246     mmu_mapPage(i * PAGE_SIZE, dir, i * PAGE_SIZE, attrs);
247 }
250 dir[0].us = 0;
```

Se mapea con Identity mapping el código y el área libre del kernel. En total ocupan 4 Mb, es decir 1024 páginas de 4 Kb, por lo que podemos resolverlo mapeando las primeras 1024 páginas del kernel. Para hacerlo utilizamos la función `mmu_mapPage`, que implementamos en la parte c. Los atributos que seteamos son los de presente, R/W y S, al tratarse de páginas del kernel. En la línea 250 ponemos la entrada del page directory como supervisor (bit U/S = 0).

```
mmu_mapPage(phys_dst_codigo_tarea,
    (page_dir_entry_t*) rcr3(),
    phys_dst_codigo_tarea,
    PAG_P | PAG_RW);
mmu_mapPage(phys_dst_codigo_tarea + PAGE_SIZE,
    (page_dir_entry_t*) rcr3(),
    phys_dst_codigo_tarea + PAGE_SIZE,
    PAG_P | PAG_RW);
```

Hacemos un identity mapping en el lugar donde vamos a copiar el código de la tarea para poder escribir en esa página.

```
uint8_t* src = (uint8_t*) addr_codigo_task_original;
uint8_t* dst = (uint8_t*) phys_dst_codigo_tarea;
for (uint32_t i = 0; i < PAGE_SIZE; ++i) {
    dst[i] = src[i];
}
mmu_unmapPage(phys_dst_codigo_tarea, rcr3());
mmu_unmapPage(phys_dst_codigo_tarea + PAGE_SIZE, rcr3());
```

Luego copiamos byte a byte desde la dirección del código original de la tarea (dentro del kernel), a la dirección de destino de la copia del código de la tarea, en el área libre de tareas. Luego desmapeamos la dirección de destino de la memoria virtual.

```
uint32_t task_code_attrs = PAG_P | PAG_US;
mmu_mapPage(TASK_CODE,
            dir,
            phys_dst_codigo_tarea,
            task_code_attrs);

mmu_mapPage(TASK_CODE + PAGE_SIZE,
            dir,
            phys_dst_codigo_tarea + PAGE_SIZE,
            task_code_attrs);
```

Mapeamos las dos páginas que comienzan en la dirección virtual TASK.CODE (0x08000000) con las dos páginas que comienzan en la dirección física de la posición en el tablero de las tareas.

```
mmu_mapPage(TASK_COMPARTIDA,
            dir,
            addr_area_compartida_jugador,
            PAG_P | PAG_RW | PAG_US);
mmu_mapPage(TASK_COMPARTIDA + PAGE_SIZE,
            dir,
            addr_area_compartida_jugador + PAGE_SIZE,
            PAG_P | PAG_RW | PAG_US);
```

Finalmente mapeamos a partir de la dirección TASK.COMPARTIDA con la dirección del área compartida de la tarea dentro del kernel.

5.3. mmu_mapPage

En esta función implementamos el mapeo de una página virtual a su dirección física. La función toma cuatro parámetros:

Nombre del parámetro	Tipo y descripción
virtual	uint32_t, dirección virtual
cr3	page_dir_entry_t*, dirección del PD
phy	uint32_t, dirección física
attrs	uint32_t, atributos del PTE

Detallamos el código de la función:

```
uint32_t pd_index = MMU_GET_DIRECTORY_INDEX(virtual);
uint32_t pt_index = MMU_GET_TABLE_INDEX(virtual);
```

Descomponemos la dirección virtual y obtenemos la PDE y la PTE correspondiente.

```
page_dir_entry_t* page_directory = MMU_GET_PAGE_DIR(cr3);
```

Limpiamos la parte baja del CR3 para alinearla a una página de la memoria física.


```
125 if ( ! page_directory[pd_index].p ) {
129     uint32_t table_phys_addr = mmu_nextFreeKernelPage();
131     page_directory[pd_index].p = 1;

135     page_directory[pd_index].rw = 1;

139     page_directory[pd_index].us = 1;
140     page_directory[pd_index].pwt = 0;
141     page_directory[pd_index].pcd = 0;
142     page_directory[pd_index].a = 0;
143     page_directory[pd_index].ign = 0;
144     page_directory[pd_index].ps = 0; // Pagina de 4k
145     page_directory[pd_index].g = 0;
146     page_directory[pd_index].avl = 0;

149     page_directory[pd_index].table_addr =
150         (table_phys_addr >> 12);

153     for (uint32_t i = 0; i < 1024; ++i) {
154         ((page_table_entry_t*) table_phys_addr)[i].p = 0;
155     }

157 }
```

Este código construye la PT en caso de no estar presente.

En la línea 129 y 131 pedimos una página al kernel para la nueva tabla. Usa una página del kernel ya que el MMU funciona a nivel 0.

En la línea 135 seteamos el atributo de R/W de la PDE en 1, para permitir mezcla de páginas de lectura/escritura y de sólo lectura. En el caso que una página sea de sólo lectura, se debe poner el bit R/W de su PTE en 0.

En la línea 139 seteamos en 1 el bit de U/S, para permitir mezcla de páginas de usuario y kernel. Si una página es de kernel, se pondrá en 0 el bit de U/S en la PTE correspondiente.

Finalmente en la línea 149 se guarda la base de la dirección física de la PT, y luego se limpia el bit de Present en todas las entradas de la PT recién creada.

```
uint32_t* page_table =
    (uint32_t*) MMU_GET_TABLE_PHYS_ADDR(page_directory[pd_index]);
page_table[pt_index] = (phy & 0xFFFFF000) | PAG_P | attrs;

tlbflush();
```

En cualquier caso, tanto si la PT estaba presente como si no, se accede a la PT dada por la PDE. Luego se guarda en la PTE correspondiente la dirección física a mapear, con los atributos pasados en el argumento, y el bit de Present seteado.

Para terminar seteamos el bit de presente en la PT y luego invalidamos la caché, ya que modificamos el esquema de paginación.

5.4. mmu_unmapPage

```
uint32_t pd_index = MMU_GET_DIRECTORY_INDEX(virtual);
uint32_t pt_index = MMU_GET_TABLE_INDEX(virtual);
```

Obtenemos la PDE y la PTE correspondientes.

```
page_dir_entry_t* dir = MMU_GET_PAGE_DIR(cr3);
page_table_entry_t* table = MMU_GET_TABLE_PHYS_ADDR(dir[pd_index]);
```

obtenemos la dirección base del PD y de la PT.

```
table[pt_index].p = 0;
tlbflush();
```

Seteamos el bit de presente en la PT y luego invalidamos la caché, ya que modificamos el esquema de paginación.

5.5. Prueba de paginación

Para probar paginación desarrollamos la parte d. del ejercicio.

```
; ; DESCOMENTAR PARA PROBAR mmu_initTaskDir
; ; Pruebo esquema de paginacion de tarea (la parte del kernel) cargando un
; ; mapa de memoria de tarea.
; push 0
; push 0
; xchg bx, bx
; call mmu_initTaskDir
; mov ebx, cr3 ; Preservo el directorio de kernel.
; mov cr3, eax ; Pone CR3 de prueba
; xchg bx, bx
; call screen_limpiar ; Limpia pantalla
; mov cr3, ebx ; Esto restaura el mapa de memoria original
; xchg bx, bx
```

6. Ejercicio 6

En el ejercicio 6 consistió en enfocarnos en las tareas propiamente dichas, creando sus TSS y poblando la GDT con descriptores de TSS.

En el archivo kernel.asm agregamos los comandos para inicializar las TSS:

```
; Inicializar tss
call tss_init
mov ax, 0xC0 ; Indice tarea idle << 3
ltr ax

; Inicializar tss de la tarea Idle
call fill_tss_idle
```

6.1. tss_init

La función `tss_init` solamente agrega a la GDT el descriptor del TSS de la tarea inicial. No hace falta inicializar los campos del TSS ya que es solamente un placeholder en memoria para que el procesador guarde el contexto al hacer el primer salto de tarea.

Veamos la función que se encarga de inicializar una entrada en la GDT para un descriptor de TSS: `tss_initGdtEntry`.

Nombre del parámetro	Tipo y descripción
ix	uint32_t, índice en la GDT.
base	tss*, puntero a la TSS a registrar en la GDT
dpl	uint32_t, indica el DPL de la tarea

La función `tss_initGdtEntry` simplemente setea en el índice `ix` pasado por parámetro, un nuevo descriptor de TSS con la base y el DPL también pasados. De la siguiente manera setea la nueva entrada de la GDT:

```

gdt[ix].limit_0_15 = TSS_SIZE;           // Tamaño de una TSS
gdt[ix].base_0_15  = (uint32_t) base & 0xFFFF; // base[15:0]
gdt[ix].base_23_16 = ((uint32_t) base >> 16) & 0xFF; // base[23:16]
gdt[ix].type       = 0b1001;
gdt[ix].s          = 0x0;                // 0 = sistema
gdt[ix].dpl        = dpl;
gdt[ix].p          = 0x1;
gdt[ix].limit_16_19 = 0x0;
gdt[ix].avl        = 0x0;
gdt[ix].l          = 0x0;
gdt[ix].db         = 0x0;
gdt[ix].g          = 0x0;
gdt[ix].base_31_24 = ((uint32_t) base >> 24);

```

Notar que inicializamos el bit de Busy en cero.

6.2. fill_tss_idle

La función `tss_idle` sí inicializa los valores de la TSS de la tarea Idle, y además inicializa la entrada de la GDT correspondiente a ella. El lugar en la GDT para el descriptor de la TSS de la tarea Idle es una constante, llamada `GDT_TSS_IDLE`. También lo es el índice a partir del cual agregamos los descriptor de las TSS de las otras tareas, llamada `GDT_TSS_INIT`.

```

#define GDT_TSS_IDLE      23
#define GDT_TSS_INIT      24

```

La función `fill_tss_idle` empieza por setear los atributos de la TSS de la tarea Idle:

```
tss_idle.eip = 0x00014000;
tss_idle.eflags = 0x00000202;
tss_idle.cr3 = KERNEL_PAGE_DIR; // Usa dir del kernel
tss_idle.esp = KERNEL_STACK;
tss_idle.ebp = KERNEL_STACK;
tss_idle.cs = GDT_OFF_CODIGO_KERNEL;
tss_idle.gs = GDT_OFF_DATOS_KERNEL;
tss_idle.fs = GDT_OFF_DATOS_KERNEL;
tss_idle.ds = GDT_OFF_DATOS_KERNEL;
tss_idle.es = GDT_OFF_DATOS_KERNEL;
tss_idle.ss = GDT_OFF_DATOS_KERNEL;
tss_idle.eax = 0x0;
tss_idle.ecx = 0x0;
tss_idle.edx = 0x0;
tss_idle.ebx = 0x0;
tss_idle.esi = 0x0;
tss_idle.edi = 0x0;
tss_idle.ss0 = GDT_OFF_DATOS_KERNEL;
tss_idle.esp0 = KERNEL_STACK;
tss_idle.iomap = 0xFFFF;
```

Como vemos, los valores de los atributos de la tarea Idle se inicializan con los valores de la tarea del kernel. Los registros de segmentos y el de código corresponden a sendos segmentos del kernel. Como se pide en la parte b del ejercicio, los registros de stack apuntan al stack del kernel, y el registro CR3 al directorio de páginas del kernel. En cuanto al Instruction Pointer, al comienzo del código de la tarea Idle, 0x00014000.

Finalmente, completamos la entrada correspondiente en la GDT.

6.3. fill_tss

La función fill_tss inicializa la TSS para un jugador. Arma también su esquema de memoria. Veamos sus parámetros:

(tss* actual, uint32_t jugador, uint32_t fila)

Nombre del parámetro	Tipo y descripción
actual	tss*, puntero a la dirección física de la TSS actual.
jugador	tssuint32_t, jugador corresp. a la tarea actual.
fila	uint32_t, fila donde se encuentra la tarea actual.

La función lo primero que hace es actualizar los atributos de la TSS actual.

```
actual->eip = TASK_CODE;
actual->cr3 = (uint32_t) mmu_initTaskDir(jugador, fila);
actual->eflags = 0x00000202; // despues va 202
actual->iomap = 0xFFFF;
actual->cs = GDT_OFF_CODIGO_USER;
actual->gs = GDT_OFF_DATOS_USER;
actual->fs = GDT_OFF_DATOS_USER;
actual->ds = GDT_OFF_DATOS_USER;
actual->es = GDT_OFF_DATOS_USER;
actual->ss = GDT_OFF_DATOS_USER;
actual->eax = 0x0;
actual->ecx = 0x0;
actual->edx = 0x0;
actual->ebx = 0x0;
actual->esi = 0x0;
actual->edi = 0x0;
actual->esp = TASK_STACK;
actual->ebp = TASK_STACK;
```

El Instruction Pointer (eip) se setea en la dirección virtual del código (0x08000000).

6.4. Salto de la tarea inicial a la tarea idle

Para realizar el cambio de la tarea inicial a la tarea idle, primero inicializo la tss de la tarea inicial

```
; Inicializar tss
call tss_init
mov ax, 0xC0 ; 0xC0 = indice tarea inicial >> 3
ltr ax
```

Luego inicializo la TSS de la tarea idle:

```
; Inicializar tss de la tarea Idle
call fill_tss_idle
```

Y luego salta a la tarea idle:

```
; Saltar a la primera tarea (Idle)
jmp 0xB8:0 ; 0xB8 = indice tarea idle >> 3
```

7. Ejercicio 7

7.1. Inicialización de las estructuras del scheduler

El scheduler mantiene el estado de cuatro propiedades:

```
uint32_t sched_player_row[] = {20, 20};
uint32_t sched_miners_left[2] = {20, 20};
uint8_t sched_last_miner[2] = {0, 0};
uint8_t sched_this_miner = 0;
```

La primera guarda la fila en donde se encuentra cada jugador, y la segunda, su cantidad de tareas disponibles para crear (veinte como máximo durante todo el juego). *sched_last_miner* guarda el índice del la última tarea minera despachada por el scheduler para cada jugador, y *sched_this_miner* guarda el índice del minero actual.

Por otra parte, en el archivo *state.h* guardamos el estado del juego en general:

```
typedef struct {
    int32_t pos_x;
    int32_t pos_y;
    uint32_t n_cristales;
    uint32_t alive;
} sched_miner_state_t;
```

Es la estructura de del estado de un minero. Su posición en las coordenadas x e y, la cantidad de cristales que comió, y si el minero está vivo o muerto.

```
sched_miner_state_t sched_miners_state[20];
```

Guardamos en un array una lista con los 20 posibles estados de mineros. Los mineros del jugador A son del 0 al 9, y los del B, del 10 al 19.

Además guardamos como variables globales las propiedades del escheduler, ya que las necesitamos para spawnear una nueva tarea.

```
extern uint32_t sched_player_row[2];
extern uint32_t sched_miners_left[2];
extern uint8_t sched_last_miner[2];
extern uint8_t sched_this_miner;
```

Volviendo a la inicialización del scheduler, tenemos la función *sched_init*:

```
void sched_init() {
    for (uint32_t i = 0; i < 20; ++i) {
        sched_miners_state[i].alive = FALSE;
    }
}
```

Lo único que hace es inicializar a todas las tareas como muertas, ya que debe esperar el input correspondiente a lanzar una tarea de un jugador.

7.2. sched_nextTask

De acuerdo al algoritmo dado, el scheduler alterna una tarea del jugador A, una tarea del jugador B.

```
uint32_t jugador_actual = (sched_this_miner < 10) ? PLAYER_A : PLAYER_B;
(jugador_actual == PLAYER_A) ? PLAYER_B : PLAYER_A;
```

Primero determina cuál es el jugador actual y cuál el otro.

```
uint16_t ix;

uint32_t hay_tarea_jugador_opuesto =
    aux_get_next_alive_miner(jugador_opuesto, &ix);
```

La función auxiliar *aux_get_next_alive_miner* determina si un jugador (en este caso el jugador opuesto), tiene un minero vivo, y en ese caso, cambia *ix* para que sea igual a ese índice. Si el jugador tiene un minero vivo, devuelve 1 (TRUE), y sino devuelve 0 (FALSE).

```
if (hay_tarea_jugador_opuesto) {
    sched_this_miner = ix;
    sched_last_miner[jugador_opuesto] = ix % 10;
    return aux_get_gdt_selector(ix);
}
```

En el caso de que el jugador opuesto tenga una tarea viva, devuelve el índice en la GDT del selector de TSS correspondiente a esa tarea. Para eso utiliza una función auxiliar, *aux_get_gdt_selector*, que toma un índice de tarea, y devuelve el índice en la GDT correspondiente.

```
uint32_t hay_tarea_este_jugador =
    aux_get_next_alive_miner(jugador_actual, &ix);
if (hay_tarea_este_jugador) {
    sched_this_miner = ix;
    sched_last_miner[jugador_actual] = ix % 10;
    return aux_get_gdt_selector(ix);
}
return GDT_TSS_IDLE << 3;
```

En el caso de que el jugador opuesto no tenga ninguna tarea viva, busca una tarea del jugador actual. En el caso de que tampoco la encuentre, salta a la tarea idle.

7.3. Interrupciones de las tareas

En el índice 47 de la GDT definimos un descriptor de Interrupt Gate. Para atender a la interrupción utilizamos el siguiente código de ASM. Esta interrupción es de usuario, por lo tanto se ejecuta en nivel de privilegio 3. Para definir el tipo de acción que pide la tarea, se pasa por el registro EAX un entero que indica si una tarea se desea mover, minar un cristal, u obtener el id de sí misma.

```
%define syscal_move_id 177788
%define syscal_take_id 310311
%define syscal_getid_id 760279
global _isr47
_isr47:
    pushad

    cmp eax, syscal_move_id
    je .move
    cmp eax, syscal_take_id
    je .take
    cmp eax, syscal_getid_id
    je .getid

    call sched_kill ; Mata esta tarea
    jmp .jmp_to_idle ; 0xB8 = ix tarea idle << 3
```

La interrupción primero determina si la interrupción corresponde a un movimiento, a tomar un cristal, o a pedir el id. Si no es ninguno de esos casos, se mata a la tarea y se salta a la tarea idle.

Luego se llama al código (implementado en C), para realizar la acción pedida por la interrupción:

```
.move:
    push ebx
    call game_move
    pop ebx
    jmp .jmp_to_idle

.take:
    call game_take
    mov [esp + 28], eax
    jmp .jmp_to_idle

.getid:
    call game_get_id
    mov [esp + 28], eax

    jmp .fin
```

7.3.1. game_move

`game_move` es una de las funciones más complejas del trabajo, dado que tiene que: calcular el offset y la nueva ubicación en el tablero, copiar el código de la tarea a la nueva ubicación, actualizar el esquema de paginación, manejar el dibujo de la pantalla y sus cristales y si es necesario, actualizar el puntaje de un jugador. Toma como input la dirección *d* en la que desea moverse la tarea.

```
    sched_miner_state_t* player = &sched_miners_state[sched_this_miner];

    int32_t mov_x = 0;
    int32_t mov_y = 0;

    if (d == Forward) {
        mov_x = 1;
    } else if (d == Back) {
        mov_x = -1;
    } if (d == Left) {
        mov_y = 1;
    } else if (d == Right) {
        mov_y = -1;
    };

    // Espeja los movimientos para el jugador de la derecha.
    if (game_get_player() == 1) {
        mov_x *= -1;
        mov_y *= -1;
    }

    int32_t nuevo_y = mod(player->pos_y + mov_y, TABLERO_N_FILS);

    if (! AUX_INRANGE_X(nuevo_x)) {
        sched_kill();
        return;
    }
```

El método primero obtiene el estado de la tarea actual, el offset de su ubicación dependiendo *d*, y

calcula su nueva ubicación en el tablero. Se decide identificar el borde superior con el borde inferior, y matar a la tarea si se sale del rango aceptable en el eje x.

```
uint32_t addr_src = GET_ADDR_TABLERO(player->pos_x, player->pos_y);
uint32_t addr_dst = GET_ADDR_TABLERO(nuevo_x, nuevo_y);

// copia los datos de las 2 pags de codigo de la tarea
copy_page(addr_src, addr_dst);
copy_page(addr_src + PAGE_SIZE, addr_dst + PAGE_SIZE);

// Actualiza el esquema de memoria de esta tarea para que el codigo apunte
// a la nueva posicion en el tablero. Tiene que tener permiso de escritura
// porque esta todo junto codigo y stack.
uint32_t task_code_attrs = PAG_RW | PAG_P | PAG_US;
page_dir_entry_t* cr3 = (page_dir_entry_t*) rcr3();
mmu_mapPage(TASK_CODE, cr3, addr_dst, task_code_attrs);
mmu_mapPage(TASK_CODE + PAGE_SIZE,
            cr3,
            addr_dst + PAGE_SIZE,
            task_code_attrs);
```

El código obtiene la dirección de fuente y destino de la tarea en el tablero, dependiendo de su ubicación actual y a la cual quiere moverse. Luego copia las dos páginas de código de la tarea en la dirección de destino.

Luego actualiza el esquema de paginación de esta tarea, porque ahora la dirección física de la tarea cambió. Para eso utiliza la función `mmu_mapPage` ya descrita en 5.3.

```
screen_draw_miner(GET_PLAYER_ID(sched_this_miner), nuevo_x, nuevo_y);
screen_redraw_crystal(player->pos_x, player->pos_y);
screen_draw_players();
```

Luego redibuja la pantalla y sus cristales con la tarea en su nueva ubicación.

```
player->pos_x = nuevo_x;
player->pos_y = nuevo_y;
```

Actualiza las coordenadas en la estructura que maneja el estado de la tarea.

```
uint32_t player_id = game_get_player();
if (nuevo_x == GET_BASE(player_id)) {
    game_puntaje[player_id] += player->n_cristales;
    player->n_cristales = 0;

    aux_check_game_over();

    screen_draw_miner_info(GET_PLAYER_ID(sched_this_miner),
                          GET_PLAYER_MINER_ID(sched_this_miner));

    screen_draw_score();
}
```

Finalmente se encarga de incrementar el puntaje del jugador actual si llegó al borde. También debe chequear si termina el juego (en el caso que uno de los jugadores llegue a 300 puntos). Además actualiza la cantidad de cristales del minero y del jugador en caso de ser necesario.

7.3.2. game_take

Este método primero obtiene las coordenadas de la tarea, se fija si hay cristales o no en esa posición, si no hay vuelve, y si hay, y la tarea puede tomarlos, lo hace. Si no, no hace nada.

```
sched_miner_state_t* player =
    &sched_miners_state[sched_this_miner];
int32_t x = player->pos_x;
int32_t y = player->pos_y;

uint32_t crystal_size = crystal_map[x][y];
```

Primero obtiene la posición de la tarea y si la cantidad de cristales en ese lugar.

```
if (crystal_size == 0) return -1;
```

Si no hay cristales, vuelve.

```
uint32_t suma = player->n_cristales + crystal_size;
if (suma <= GAME_MAX_CRYSTALS) {
    // Decrementa cristales en el mapa
    crystal_map[x][y] = 0;
    // Incrementa cantidad de cristales en el jugador.
    player->n_cristales = suma;

    // Imprime el estado del minero en pantalla.
    screen_draw_miner_info(GET_PLAYER_ID(sched_this_miner),
                           GET_PLAYER_MINER_ID(sched_this_miner));

    return crystal_size;
} else {
    return 0; // No se mino nada.
}
```

Si hay, se fija si la tarea tiene capacidad de tomar ese cristal. En caso afirmativo, setea los cristales de esa posición en cero, incrementa la cantidad de cristales del jugador, e imprime el estado en pantalla.

7.3.3. game_get_id

La función toma el task register, los shiftea tres bits a la derecha para obtener el índice en la GDT de la tarea. Luego le resta la base del comienzo de los índices de las tareas en la GDT (GDT_PLAYER_TASK_BASE= 25). Además toma módulo 10 y suma uno porque los índices de las tareas están entre 1 y 10.

7.4. Intercambio de tareas

En la interrupción 32 implementamos la interrupción del reloj. Tenemos que tener en cuenta el modo debug, en caso de encontrarse en modo debug el scheduling de tareas se omite. Sino, llama a la función sched_nextTask, que retorna en ax el selector de la próxima tarea. El código chequea además que no se salte a la misma tarea. Si todo anda bien, hace un jmp far a la siguiente tarea scheduleada.

7.5. Atención de excepciones

La rutina de atención para las excepciones se encarga de matar a la tarea que la origina.

Para eso primero llama a la función *idt_print_mensaje_error*, que dependiendo de si estamos en modo debug o no, muestra el estado de los registros y otra información relevante para debugear. La función *idt_print_mensaje_error* toma como parámetros el código de la interrupción que generó la excepción, y un struct **estado** de tipo *idt_exception_context_t*. El estado se construye pusheando manualmente todos los selectores de segmento del contexto actual a el stack, y los otros registros se pushean al principio de la rutina de atención con *pushad*. El método *idt_print_mensaje_error* solamente muestra en pantalla los registros si se está en modo debug, sino retorna.

Finalmente, la rutina de atención saca los parámetros del stack que pushee anteriormente, y llama a la función *sched_kill*.

```
void sched_kill() {
    sched_miners_state[sched_this_miner].alive = FALSE;

    gdt[GDT_PLAYER_TASK_BASE + AUX_GET_GDT_IX(sched_this_miner)].p = 0;
```

Esta función deberá "matar" a la tarea. Es decir, marcar su estado *.alive* como FALSE, poner en 0 el bit de presente del selector de TSS correspondiente a esa tarea.

```
// Limpia la posicion del tablero del minero.
uint32_t x = sched_miners_state[sched_this_miner].pos_x;
uint32_t y = sched_miners_state[sched_this_miner].pos_y;
screen_redraw_crystal(x, y);

// Actualiza el estado del minero en pantalla
screen_draw_miner_info(GET_PLAYER_ID(sched_this_miner),
                       GET_PLAYER_MINER_ID(sched_this_miner));
screen_draw_players();
}
```

Luego de limpiar las estructuras, debe limpiar la pantalla.

7.5.1. debugging

El método de debugging utiliza dos estructuras: *idt_exception_context_t*, ya explicada en la sección anterior, e *idt_exception_context2_t*, que contiene los valores pusheados automáticamente antes de iniciarse la ISR:

```
typedef struct {
    uint32_t eip;
    uint16_t cs;
    uint32_t eflags;
    uint32_t esp;
    uint32_t ss;
} idt_exception_context2_t;
```

El estado del stack en el punto de llamar a la función *idt_print_mensaje_error* es:

eax
esp
ss
gs
fs
es
ds
cs
edi
esi
ebp
eip
cs
eflags
esp
ss

Donde los valores ss-eip fueron pusheados automáticamente antes de comenzar a ejecutarse la ISR, luego están los valores pusheados por el `pushad`, y luego los selectores de segmento pusheados automáticamente. Notar que segundo desde arriba de todo está `esp` porque la función toma como segundo argumento un *puntero* a contexto, no un contexto directamente.

La función debug para printear los valores de las flags, el cs, eip, el esp y el ss utiliza la estructura `idt_exception_context2_t`, y para accederla, le suma el tamaño de la estructura `idt_exception_context_t`:

```
idt_exception_context2_t* estado2 =  
    ((idt_exception_context2_t*) estado) + sizeof(idt_exception_context_t);
```

Para imprimir en pantalla los valores utilizamos la función `print_hex`, por ejemplo:

```
print("ss", 22, 20, color);  
print_hex(estado2->ss, 4, 29, 20, color);
```

Al final, imprimimos los 5 valores superiores del stack:

```
uint32_t* stack_base = (uint32_t*) estado2->esp;  
print("stack", 22, 29, color);  
for (uint32_t i = 0; i < 5; ++i) {  
    uint32_t* dato_stack =  
        (uint32_t*) stack_base + i * sizeof(uint32_t*);  
    print_hex(*dato_stack, 8, 25, 30 + i, color);  
}
```

8. Despacho de tareas

En el archivo `game.c` incluimos la mayor parte de los métodos propios del juego.

La función `game_move_player` se encarga del movimiento de un jugador. Una de las decisiones que tomamos es wrappear el desplazamiento vertical, identificando los extremos. Así, cuando un jugador llega a la fila cero y se mueve para arriba, vuelve a la última fila.

Cuando un jugador quiere despachar un minero mediante, la rutina de atención para la interrupción del teclado llama a la función `idt_int_teclado` que a su vez invoca a `sched_spawn` de `sched.c`.

En esta función hacemos el chequeo de que el jugador pueda efectivamente despachar un nuevo minero. Si es así inicializa las estructuras correspondientes para la tarea: su TSS, la su entrada correspondiente en la GDT. Luego actualiza el estado de la estructura del scheduler.

```
// Completa una TSS para la tarea nueva.
tss* new_tss = (tss*) mmu_nextFreeKernelPage();
fill_tss(new_tss, player_id, sched_player_row[player_id]);
// Completa el descriptor de la GDT para la nueva TSS.
tss_initGdtEntry(gdt_ix, new_tss, 3);

// Inicializa el estado del scheduler.
sched_miners_state[slot].pos_x = (player_id == PLAYER_A) ? 0 : 77;
sched_miners_state[slot].pos_y = sched_player_row[player_id];
sched_miners_state[slot].n_cristales = 0;
sched_miners_state[slot].alive = TRUE;
```

Por último dibuja el minero en la pantalla junto a su información.

8.0.1. Dibujo de pantalla

Finalmente, para mostrar la pantalla decidimos modularizar el dibujo de los distintos objetos del juego. Así, tenemos distintos métodos para dibujar cajas de color, dibujar distintas pantallas como por ejemplo la pantalla que aparece si un jugador gana, dibujar los jugadores, los cristales, etc.

Todas estas funciones se encuentran en **screen.c**.